

Python Course for Physicists (And everyone else!)

Chunk 11: Reading and Writing Files

By Jonathan Bollig

10.10.2025

Focus: Delimiter Separated Values (e.g. csv)

- “Normal” way:

```
with open(filename) as f:  
    s = f.read()
```

→ Slow

→ No conversion to numpy data

- Better way, works 90 % of time: `np.loadtxt(filename)`

→ Lot of helpful options

→ Generates a numpy array

→ BUT: does not allow “,” as a decimal separator:

✗ 3,1415

- Solution: Pandas



```
1880.001,-0.692  
1880.004,-0.592  
1880.007,-0.673  
1880.010,-0.615  
1880.012,-0.681  
1880.015,-0.743  
1880.018,-0.646  
1880.021,-0.716  
1880.023,-0.984  
1880.026,-0.937  
1880.029,-1.125  
1880.031,-1.283  
1880.034,-1.108  
1880.037,-0.960  
1880.040,-0.865  
1880.042,-0.753  
1880.045,-0.511  
1880.048,-0.678  
1880.051,-0.766  
1880.053,-0.875  
1880.056,-0.913  
1880.059,-0.812  
1880.062,-0.744  
1880.064,-0.762  
1880.067,-0.900  
1880.070,-1.070  
1880.072,-1.260
```

Pandas



- Library for tabular data (built on top of numpy)
 - Can be very useful, but out of scope for this course
 - We only use it for reading data
- Install in Anaconda Powershell Prompt:
conda install pandas
- To read a table:

```
filename = "data\\daily_temperature_cleaned.txt"
data = pd.read_table(filename, delimiter=',').to_numpy()
print(data)
```

```
[[ 1.880004e+03 -5.920000e-01]
 [ 1.880007e+03 -6.730000e-01]
 [ 1.880010e+03 -6.150000e-01]
 ...
 [ 2.022574e+03  1.574000e+00]
 [ 2.022577e+03  1.577000e+00]
 [ 2.022579e+03  1.629000e+00]]
```

```
1880.001,-0.692
1880.004,-0.592
1880.007,-0.673
1880.010,-0.615
1880.012,-0.681
1880.015,-0.743
1880.018,-0.646
1880.021,-0.716
1880.023,-0.984
1880.026,-0.937
1880.029,-1.125
1880.031,-1.283
1880.034,-1.108
1880.037,-0.960
1880.040,-0.865
1880.042,-0.753
1880.045,-0.511
1880.048,-0.678
1880.051,-0.766
1880.053,-0.875
1880.056,-0.913
1880.059,-0.812
1880.062,-0.744
1880.064,-0.762
1880.067,-0.900
1880.070,-1.070
1880.072,-1.260
```

Relative and Absolute Paths

- Path: String that specifies a file / directory location:
- Absolute Path: Path from “the bottom”:
`C:\2025 Python Course\Task Sheets\data\oscilloscope.csv`
- Current Working Directory (cwd)
 - cwd: The file path where your code is executed: Usually the parent folder of your executed file:
`C:\2025 Python Course\Task Sheets\`
 - Find out the cwd:

```
import os  
print(os.getcwd())
```
- Relative Path: Path relative to the current working directory:
`data\oscilloscope.csv`
- File names can be given as a relative or absolute path.
- Careful: “\” is an escape sequence in a string → use “\\” instead.

Live Coding: The Try and Error of File Loading

- Data is often “dirty”:
 - Weird formats
 - Missing values
 - More data than we need
 - Headers and footers
 - Much more stuff, that messes it up
- → Usually, a try and error process:
 - No universal method ☹️

```
% Results are based on 50461 monthly time series
%   with 21047039 observations and 48263 daily time series
%   with 512331899 observations
%
% Estimated Jan 1951-Dec 1980 land-average temperature (C): 8.59 +/- 0.05
%
%
% Date Number, Year, Month, Day, Day of Year, Anomaly
1880.001      1880      1      1      1      -0.692
1880.004      1880      1      2      2      -0.592
1880.007      1880      1      3      3      -0.673
1880.010      1880      1      4      4      -0.615
1880.012      1880      1      5      5      -0.681
1880.015      1880      1      6      6      -0.743
1880.018      1880      1      7      7      -0.646
1880.021      1880      1      8      8      -0.716
1880.023      1880      1      9      9      -0.984
1880.026      1880      1     10     10     -0.937
1880.029      1880      1     11     11     -1.125
1880.031      1880      1     12     12     -1.283
1880.034      1880      1     13     13     -1.108
```

File Saving

- Human readable: Easy with numpy:

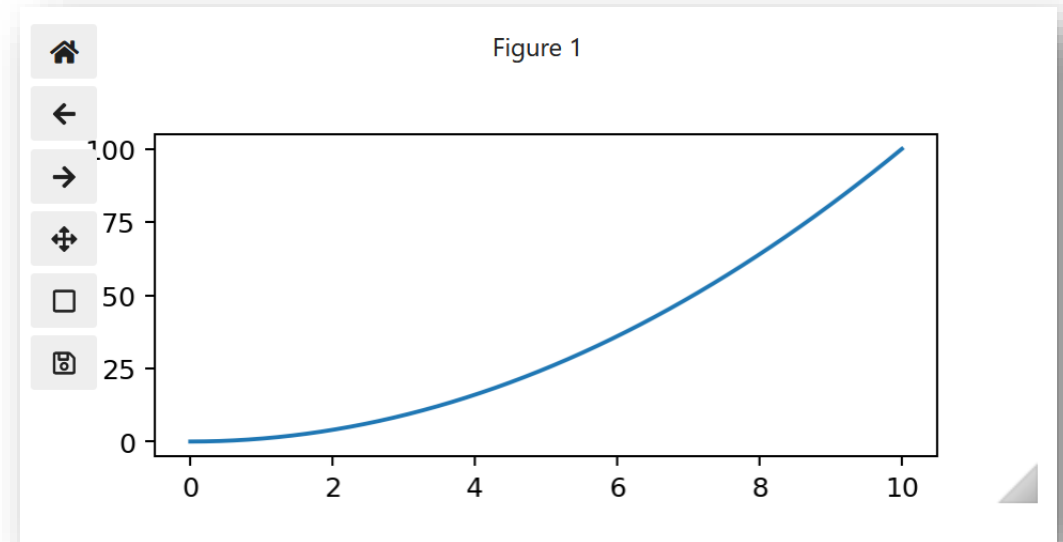
```
data = pd.read_table(load_filename, sep='\\s+', skiprows=22).to_numpy()[ :, [0, 5]]
save_filename = "data\\daily_temperature_cleaned.txt"
np.savetxt(save_filename, data, fmt='%.3f', delimiter=",")
```

- Only machine readable:
 - Faster
 - Error proof

```
np.save("data", data)
data = np.load("data.npy")
```

Extra Tip: Interactive Matplotlib

- Matplotlib can be made interactive:
- For this: `conda install ipympl`



- Before importing: `%matplotlib widget`

```
%matplotlib widget
import matplotlib.pyplot as plt
```

Good Luck with your Tasks 😊

Morning: Program works perfectly, no errors

Program in the afternoon:

